NUMERICAL SIMULATION OF A SPRING-CONNECTED TETRAHEDRON IN A THREE-DIMENSIONAL BOX

Energy Transfer from Centre-of-Mass Motion to Internal Vibrations During Elastic Collisions

A Project in the Field of Numerical Methods in Classical Field Theory and Quantum Mechanics for the Degree of Master of Science in Computer Simulation in Science

Summer Semester 2026

Bergische Universität Wuppertal

Submitted by:	**Submitted to:**
Sunzid Ahmed Rafi Matriculation Number: 2577989	Prof. Szabolcs Borsányi

Copyright 2026 - Sunzid Ahmed Rafi

Table of Content

1. Introduction....3
1.1 Background and Physical Idea....3
2. Physical Theory....3
2.1 Tetrahedron and Spring Model....3
2.2 Energy of the System....3
3. Numerical Method and Implementation....4
3.1 Spring Force Calculation....4
3.2 Velocity Verlet Integration....4
3.3 Event-Based Collision Detection....4
3.4 Elastic Reflection....4
3.5 Centre-of-Mass and Rotational Energy....4
3.6 Vibrational and Total Energy....5
4. Numerical Experiments and Main Results....5
4.1 Wall Collision....5
4.2 Off-Centre Collision with the Rigid Sphere....5
4.3 Numerical Result Summary....6
5. Energy Conservation and Numerical Accuracy....6
6. Discussion and Numerical Experience....7
7. Conclusion....7
7.1 Code and Supplementary Material....7
References....7
AI Assistance Disclosure....7

Copyright 2026 - Sunzid Ahmed Rafi

1. Introduction

1.1 Background and Physical Idea

A deformable object can move as a whole and also vibrate internally. When such an object collides with a wall or another rigid body, different parts may change their motion at different times. Part of the translational motion can therefore be converted into internal vibration. This transfer of energy is the central physical idea of this project.

The model consists of four equal-mass particles placed at the vertices of a regular tetrahedron. Its six edges are represented by identical harmonic springs. For a spring joining particles i and j, the instantaneous length is $L_{ij} = \left|\overline{x_j} - \overline{x_i}\right|$ and the elastic potential energy is

$$V_{ij} = \frac{1}{2}k(|\overline{x_j} - \overline{x_i}| - a)^2 ,$$

where a is the equilibrium spring length and k is the spring constant. Initially all six springs have length a and all four particles receive the same velocity, so the tetrahedron begins in pure translational motion with essentially no internal vibration or rotation.

The tetrahedron is placed inside a three-dimensional cubic box. In the first experiment it collides with the box walls. In the second experiment a rigid sphere is placed at the centre of the box and the tetrahedron hits it off-centre. The curved obstacle makes it possible to generate rotation as well as vibration.

1.2 Aim of the Numerical Study

The objective is to follow the motion of all four particles and determine how centre-of-mass kinetic energy is redistributed after collisions. The centre-of-mass kinetic energy is

$$E_{COM} = \frac{1}{2}M|\overline{V}_{COM}|^2, \quad M = 4m .$$

The equations are solved numerically in C++. Particle positions, velocities and energy components are written to CSV files and analysed with Python. The numerical results are presented through energy curves, spring-deformation plots, particle trajectories and three supplementary animations. A second objective is to verify that the total mechanical energy is conserved to high numerical accuracy.

2. Physical Theory

2.1 Tetrahedron and Spring Model

The tetrahedron contains four particles of mass m and six identical springs. For two connected particles, $d = x_j - x_i$ and L = |d|. Differentiating the harmonic potential gives the restoring force [3]

$$\overline{F}_{i\leftarrow j} = k(L - a)\frac{d}{L}, \quad \overline{F}_{j\leftarrow i} = -\overline{F}_{i\leftarrow j} .$$

The initial coordinates are chosen as a regular tetrahedron, so every edge length equals a and the initial spring energy is zero. The six spring pairs are (0,1), (0,2), (0,3), (1,2), (1,3) and (2,3).

2.2 Energy of the System

The kinetic and spring energies are

$$E_{kinetic} = \sum_i \frac{1}{2}M|\overline{v}_i|^2, \quad E_{spring} = \sum_{six\ spring} \frac{1}{2}k(L_{ij} - a)^2 .$$

The total mechanical energy is therefore

$$E_{Total} = E_{kinetic} + E_{spring} .$$

For the centre of mass,

$$R_{COM} = \frac{1}{M}\sum_{i=0}^{3} m_i x_i, \quad V_{COM} = \frac{1}{M}\sum_i m_i v_i .$$

To distinguish rigid rotation from internal vibration, the particle positions and velocities relative to the centre of mass are $q_i = x_i - R_{COM}$ and $u_i = v_i - v_{COM}$. The internal angular momentum is $L = \sum_i mq_i \times u_i$. The inertia tensor I is constructed from q_i and the instantaneous angular velocity is obtained from Iω = L. The rotational energy is $E_{rot} = \frac{1}{2}\omega \cdot L$.

After removing translation and the best-fit rigid rotation, the residual velocity is $u_{vib,i} = u_i - \omega \times q_i$. The vibrational energy used in the analysis is

$$E_{vib,kin} = \sum_i \frac{1}{2}m\left|u_i - \omega \times q_i\right|^2 .$$

This gives the numerical decomposition $E_{Total} \approx E_{COM} + E_{rot} + E_{vib}$, which is checked directly from the simulation output.

Copyright 2026 - Sunzid Ahmed Rafi

3. Numerical Method and Implementation

3.1 Spring Force Calculation

Spring forces are recalculated from the current particle positions at every time step. Equal and opposite forces are accumulated on the two particles of each spring.

```
void calculateForces(Particle p[], const Spring springs[], double k, double a) {
    for (int i=0;i<N;i++) p[i].f = {0,0,0};
    for (int s=0;s<NS;s++) {
        int i = springs[s].i, j = springs[s].j;
        Vec3 d = p[j].r - p[i].r;
        double L = norm(d);
        if (L < EPS) continue;
        Vec3 force = d * (k * (L-a) / L);
        p[i].f += force;  p[j].f -= force;
    }
}
```

Code Listing 1. Calculation of the six harmonic spring forces.

3.2 Velocity Verlet Integration

Newton's equations are integrated with the Velocity Verlet scheme, which is commonly used for conservative particle systems because of its favourable energy behaviour [1,2]. A time step $\Delta t = 10^{-4}$ is used.

```
// half kick
for (int i=0;i<N;i++) p[i].v += p[i].f * (0.5*dt/p[i].m);
// drift and resolve exact collision events
for (int i=0;i<N;i++)
    driftParticleWithCollisions(p[i], dt, box, sphere, t, summary.collisions);
// new spring forces
calculateForces(p, springs, k, a);
// second half kick
for (int i=0;i<N;i++) p[i].v += p[i].f * (0.5*dt/p[i].m);
```

Code Listing 2. Main Velocity Verlet integration cycle.

3.3 Event-Based Collision Detection

A simple overshoot correction can introduce significant error when a particle crosses a boundary during one time step. The program therefore searches for the earliest collision inside the current drift interval. For the central sphere, $\left|r_0 + vt - r_s\right|^2 = R^2$ gives a quadratic equation for the collision time.

```
Vec3 r = p.r - sphere.center;
double A = norm2(p.v);
double B = 2.0 * dot(r, p.v);
double C = norm2(r) - sphere.radius*sphere.radius;
double D = B*B - 4.0*A*C;
if (D >= 0.0) {
    double sqrtD = sqrt(D);
    double t1 = (-B - sqrtD)/(2.0*A);
    double t2 = (-B + sqrtD)/(2.0*A);
    // select earliest valid t in [0, remaining]
}
```

Code Listing 3. Determination of a particle-sphere collision time.

3.4 Elastic Reflection

After moving exactly to the collision point, the normal velocity is reflected. For the sphere, n is the outward surface normal and $v' = v - 2(v \cdot n)n$.

```
Vec3 n = normalized(p.r - sphere.center);
double vn = dot(p.v, n);
p.v -= n * (2.0*vn);
p.r += n * 1e-12;
```

Code Listing 4. Elastic reflection from the rigid sphere. Box-wall collisions similarly reverse only the normal velocity component.

3.5 Centre-of-Mass and Rotational Energy

The program builds the full symmetric inertia tensor and solves Iω = L. The symmetric off-diagonal assignments are important because the 3×3 solver uses the complete matrix.

```
Vec3 R = centerOfMass(p), V = centerOfMassVelocity(p);
E.comKinetic = 0.5 * M * norm2(V);
double I[3][3] = {{0,0,0},{0,0,0},{0,0,0}};
Vec3 L{0,0,0};
for (int i=0;i<N;i++) {
    Vec3 q = p[i].r - R, u = p[i].v - V;
    double q2 = norm2(q), m = p[i].m;
    I[0][0] += m*(q2-q.x*q.x); I[1][1] += m*(q2-q.y*q.y);
```

Copyright 2026 - Sunzid Ahmed Rafi

```
    I[2][2] += m*(q2-q.z*q.z);
    I[0][1] -= m*q.x*q.y; I[1][0] = I[0][1];
    I[0][2] -= m*q.x*q.z; I[2][0] = I[0][2];
    I[1][2] -= m*q.y*q.z; I[2][1] = I[1][2];
    L += cross(q,u) * m;
}
Vec3 omega{0,0,0}; solve3x3(I, L, omega);
E.rotational = 0.5 * dot(omega, L);
```

Code Listing 5. Centre-of-mass and rotational-energy calculation, including the symmetric inertia tensor.

3.6 Vibrational and Total Energy

```
E.vibrationKinetic = 0.0;
for (int i=0;i<N;i++) {
    Vec3 q = p[i].r - R, u = p[i].v - V;
    Vec3 uVib = u - cross(omega, q);
    E.vibrationKinetic += 0.5*p[i].m*norm2(uVib);
}
E.vibrationTotal = E.vibrationKinetic + E.spring;
E.total = E.kinetic + E.spring;
```

Code Listing 6. Vibrational and total mechanical energy.

The simulation was implemented in C++. At regular intervals, the particle positions, centre-of-mass coordinates and different energy components were written to CSV files. The C++ program generated wall_experiment.csv and sphere_experiment.csv. These files were then processed with Python using NumPy, Pandas and Matplotlib to create the plots, three-dimensional trajectories and animations used in the numerical analysis.

4. Numerical Experiments and Main Results

Two deterministic experiments were run with the same model parameters. The values below are taken directly from the C++ implementation used to generate the submitted CSV files.

Parameter	Wall experiment	Sphere experiment
m, k, a	1, 10, 1	1, 10, 1
Box	[-3,3]^3	[-3,3]^3
Time step Δt	10^-4	10^-4
Initial COM	(-1.5, 0, 0)	(-2.0, 0.45, 0.25)
Initial velocity	(1.2, 0, 0)	(1.0, 0, 0)
Sphere	disabled	R = 0.75 at origin
Simulation time	8.0	4.0

4.1 Wall Collision

The tetrahedron begins with E_{COM} = 2.88 and essentially zero internal energy. The first wall collision occurs at t ≈ 3.455. During the strongest interaction the centre-of-mass energy becomes nearly zero while the vibrational energy rises to approximately 2.88. Rotational energy remains zero because the setup is symmetric.

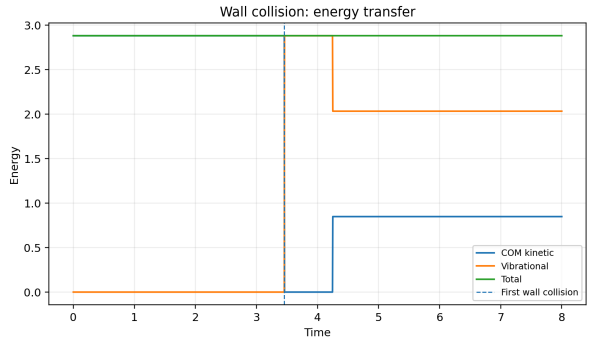


Figure 1. Centre-of-mass, vibrational and total energy during the wall experiment.

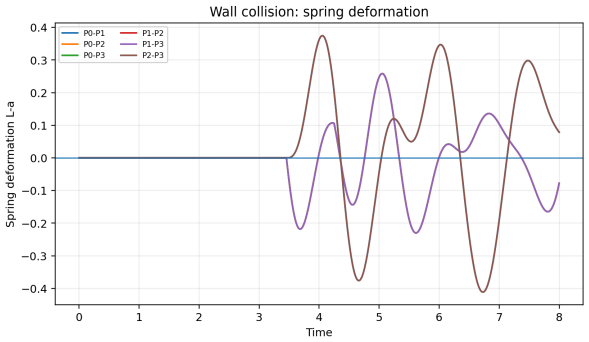


Figure 2. Deformation L-a of all six springs. Several curves overlap because of the symmetry of the wall collision.

4.2 Off-Centre Collision with the Rigid Sphere

The sphere experiment begins with total energy 2.00. The first sphere contact occurs at t ≈ 0.910. Because the impact is off-centre, both internal vibration and rigid-body rotation are excited.

Copyright 2026 - Sunzid Ahmed Rafi

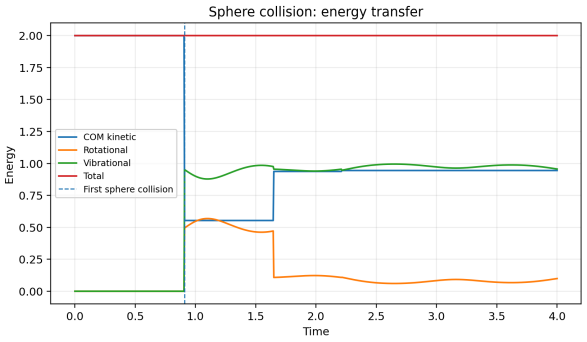


Figure 3. Energy transfer during the sphere experiment.

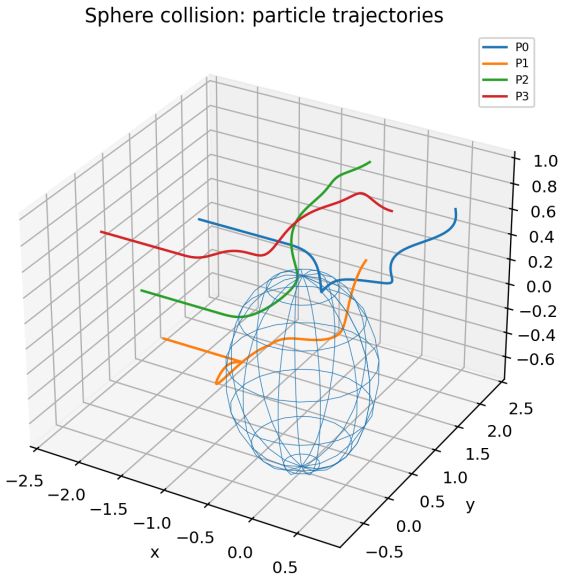


Figure 4. Three-dimensional trajectories of the four particles around the central rigid sphere.

At t ≈ 1.10, the recorded values are $E_{COM} \approx 0.553$, $E_{rot} \approx 0.569$ and $E_{vib} \approx 0.878$. Their sum is approximately 2.00, showing that the initial translational energy is redistributed rather than lost. The particle trajectories show the change in direction caused by the curved obstacle and directly satisfy the trajectory requirement of the project.

4.3 Numerical Result Summary

Quantity	Wall	Sphere
Initial total energy	2.880000	2.000000
Final total energy	2.880078	1.999933
Maximum vibration energy	2.880000	0.994805
Maximum rotation energy	0	0.568633
Maximum relative error	2.72×10^-5	3.46×10^-5
Recorded collision hits	4 wall	3 sphere

The wall experiment therefore demonstrates mainly E_{COM} → E_vibration, whereas the off-centre sphere collision demonstrates $E_{COM} \rightarrow E_{rotation} + E_{vibration}$.

5. Energy Conservation and Numerical Accuracy

Energy conservation is the main numerical validation of the simulation. The physical model contains only conservative springs and perfectly elastic fixed boundaries, so changes in the separate energy components should not produce a significant change in E_total.

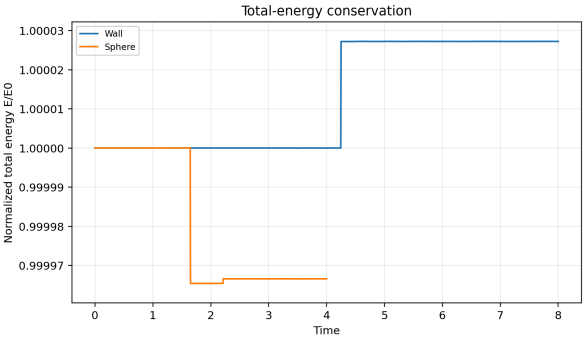


Figure 5. Normalized total energy E/E0. The y-axis is shown explicitly around one, without offset notation.

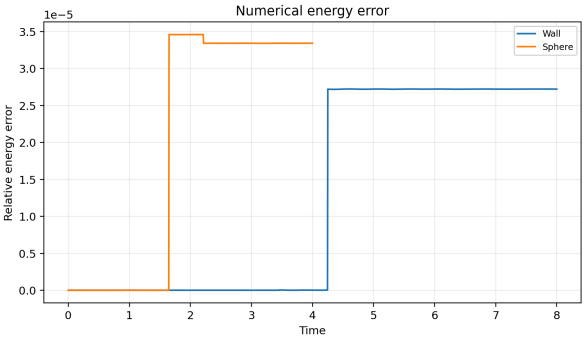


Figure 6. Relative energy error for both experiments.

For the wall experiment, E_{Total} changes from 2.880000 to 2.880078, with a maximum relative error of 2.72×10^-5 (0.00272%). For the sphere experiment, it changes from 2.000000 to 1.999933, with a maximum relative error of 3.46×10^-5 (0.00346%).

$$\epsilon_E(t) = \left| \frac{E(t) - E_0}{E_0} \right| .$$

Copyright 2026 - Sunzid Ahmed Rafi

The energy bookkeeping was also checked numerically: E_{Total} and $E_{COM} + E_{rot} + E_{vib}$ agree to floating-point precision in the saved data. The small drift in total energy is therefore a numerical integration error rather than an error in the energy decomposition.

6. Discussion and Numerical Experience

This project was more difficult than I initially expected because several numerical problems had to work correctly at the same time: three-dimensional particle motion, six changing spring forces, collision detection, separation of translational/rotational/vibrational energy and energy-conservation checks.

The collision treatment was the hardest part. A simple test performed only after a particle crossed a wall or sphere could leave the particle in an unphysical position and produced larger energy changes. I therefore used event-based collision handling: the program finds the earliest collision time inside the drift, moves the particle exactly to the contact point, reflects the normal velocity and then completes the remaining part of the time step. The sphere was harder than a flat wall because its surface normal changes at every contact point.

The second major challenge was the energy decomposition. Computing total kinetic and spring energy was straightforward, but separating centre-of-mass motion, rotation and vibration required the centre of mass, angular momentum, a full inertia tensor and a 3×3 linear solve for the angular velocity. This made the physical interpretation much clearer: a decrease in centre-of-mass energy is not a loss of energy; it appears as vibration or rotation.

Energy conservation became my main debugging tool. An animation can look physically reasonable even when the numerical method is inaccurate, so I monitored E_{Total} and the relative error throughout each run. The small step $\Delta t=10^{-4}$ was selected to keep the error below about 3.5×10^-5 in both experiments.
Finally, the plots and animations helped connect the equations to the actual motion. The symmetric wall case produced almost pure vibration, while the off-centre sphere case clearly generated rotation. The most important lesson from the project was that a reliable numerical simulation requires not only implementing equations of motion, but also validating the event handling, energy bookkeeping and numerical accuracy independently.

7. Conclusion

A spring-connected tetrahedron consisting of four particles and six harmonic springs was simulated in a three-dimensional box using Velocity Verlet integration and event-based elastic collisions. The wall experiment converted almost all centre-of-mass kinetic energy into vibration while producing essentially no rotation. The off-centre sphere experiment converted translational energy into both vibration and rotation.

The maximum relative energy errors were 2.72×10^-5 for the wall experiment and 3.46×10^-5 for the sphere experiment. These results show that the chosen integration and collision treatment conserve mechanical energy to high numerical accuracy and reproduce the expected redistribution

$$E_{COM} \rightarrow E_{vibration} + E_{rotation}.$$

7.1 Code and Supplementary Material

The source archive contains main.cpp, wall_experiment.csv, sphere_experiment.csv, generate_plots.py and generate_animations.py. The plotting script produces PNG figures only. The animation script creates three MP4 files: the wall collision, the sphere collision and a combined sphere-motion/energy visualization. Full source code is submitted separately; only six critical excerpts are reproduced in Section 3.

References

[1] L. Verlet, “Computer Experiments on Classical Fluids. I. Thermodynamical Properties of Lennard-Jones Molecules,” Physical Review, 159, 98-103 (1967).

[2] W. C. Swope, H. C. Andersen, P. H. Berens and K. R. Wilson, “A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules,” Journal of Chemical Physics, 76, 637-649 (1982).

[3] J. R. Taylor, Classical Mechanics. University Science Books (2005).

AI Assistance Disclosure

AI tools were used to assist with plot/animation generation, explanation of difficult code sections and language editing. The simulation dynamics, numerical outputs and final reported values were checked against the C++ implementation and generated CSV data.

Copyright 2026 - Sunzid Ahmed Rafi